AutoNAT 사용 예제

개요

이 문서는 Rust libp2p의 AutoNAT 프로토콜을 실제로 사용하는 다양한 예제를 제공합니다. 기본적인 클라이언트-서버 설정부터 고급 설정까지 단계별로 설명합니다.

기본 AutoNAT 클라이언트

1. 의존성 설정

```
[dependencies]

libp2p = { version = "0.53", features = ["autonat", "identify", "tcp", "noise",

tokio = { version = "1.0", features = ["full"] }

tracing = "0.1"

tracing-subscriber = "0.3"

clap = { version = "4.0", features = ["derive"] }
```

2. 기본 클라이언트 구현

```
use std::{error::Error, net::Ipv4Addr, time::Duration};

use libp2p::{

autonat,
```

```
core::{multiaddr::Protocol, Multiaddr},
identify, identity, noise,
swarm::{NetworkBehaviour, SwarmEvent},
tcp, yamux, PeerId,
};
use tracing_subscriber::EnvFilter;


#[derive(NetworkBehaviour)]
struct Behaviour {
identify: identify::Behaviour,
auto_nat: autonat::Behaviour,
}


impl Behaviour {
fn new(local_public_key: identity::PublicKey) -> Self {
Self {
identify: identify::Behaviour::new(identify::Config::new(
"/autonat-example/1.0.0".into(),
local_public_key.clone(),
)),
auto_nat: autonat::Behaviour::new(
local_public_key.to_peer_id(),
autonat::Config::default(),
```

```
),
}
}
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
tracing_subscriber::fmt()
.with_env_filter(EnvFilter::from_default_env())
.init();

let mut swarm = libp2p::SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default,
)?
.with_behaviour(|key| Behaviour::new(key.public()))?
.build();

// 로컬 주소에서 리스닝 시작
swarm.listen_on(
```

```
Multiaddr::empty()
.with(Protocol::Ip4(Ipv4Addr::UNSPECIFIED))
.with(Protocol::Tcp(0)),
)?;


// 알려진 AutoNAT 서버 추가
let server_peer_id: PeerId = "12D3KooWExample...".parse()?;
let server_address: Multiaddr = "/ip4/192.168.1.100/tcp/4001".parse()?;
swarm
.behaviour_mut()
.auto_nat
.add_server(server_peer_id, Some(server_address));


// 이벤트 루프
loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("로컬 주소에서 리스닝 중: {address}");
}
SwarmEvent::Behaviour(BehaviourEvent::AutoNat(event)) => {
handle_autonat_event(event);
}
SwarmEvent::Behaviour(BehaviourEvent::Identify(event)) => {
```

```
println!("Identify 이벤트: {event:?}");
}
event => {
println!("기타 이벤트: {event:?}");
}
}
}
}

fn handle_autonat_event(event: autonat::Event) {
match event {
autonat::Event::InboundProbe(probe_event) => {
match probe_event {
autonat::InboundProbeEvent::Request { probe_id, peer, addresses } => {
println!("인바운드 프로브 요청 받음 - ID: {probe_id:?}, 피어: {peer}, 주소: {address
}
autonat::InboundProbeEvent::Response { probe_id, peer, address } => {
println!("인바운드 프로브 응답 - ID: {probe_id:?}, 피어: {peer}, 주소: {address}");
}
autonat::InboundProbeEvent::Error { probe_id, peer, error } => {
println!("인바운드 프로브 오류 - ID: {probe_id:?}, 피어: {peer}, 오류: {error:?}");
}
}
```

```
}
autonat::Event::OutboundProbe(probe_event) => {
match probe_event {
autonat::OutboundProbeEvent::Request { probe_id, peer } => {
println!("아웃바운드 프로브 요청 전송 - ID: {probe_id:?}, 서버: {peer}");
}
autonat::OutboundProbeEvent::Response { probe_id, peer, address } => {
println!("아웃바운드 프로브 성공! - ID: {probe_id:?}, 서버: {peer}, 공개 주소: {addre
}
autonat::OutboundProbeEvent::Error { probe_id, peer, error } => {
println!("아웃바운드 프로브 실패 - ID: {probe_id:?}, 서버: {peer:?}, 오류: {error:?}
}
}
}
autonat::Event::StatusChanged { old, new } => {
println!("NAT 상태 변경: {old:?} -> {new:?}");
match new {
autonat::NatStatus::Public(addr) => {
println!("🎉 공개 주소로 접근 가능: {addr}");
}
autonat::NatStatus::Private => {
println!("🔒 NAT 뒤에 있음 (사설 네트워크)");
}
```

```
autonat::NatStatus::Unknown => {
println!("❓ NAT 상태 알 수 없음");
}
}
}
}
}
```

고급 AutoNAT 클라이언트 설정

1. 커스텀 설정을 사용한 클라이언트

```
use std::time::Duration;

fn create_custom_autonat_client(local_public_key: identity::PublicKey) -> autona
let config = autonat::Config {
// 요청 타임아웃 (기본: 30초)
timeout: Duration::from_secs(15),
// 초기 프로브 지연 (기본: 15초)
boot_delay: Duration::from_secs(5),
// 재시도 간격 - 낮은 신뢰도일 때 (기본: 90초)
retry_interval: Duration::from_secs(30),
// 갱신 간격 - 높은 신뢰도일 때 (기본: 15분)
```

```
refresh_interval: Duration::from_secs(300), // 5분

// 서버 스로틀링 기간 (기본: 90초)

throttle_server_period: Duration::from_secs(60),

// 연결된 피어를 서버로 사용 (기본: true)

use_connected: true,

// 최대 신뢰도 (기본: 3)

confidence_max: 5,

// 서버 측 설정

max_peer_addresses: 10,

throttle_clients_global_max: 20,

throttle_clients_peer_max: 2,

throttle_clients_period: Duration::from_secs(2),

// 글로벌 IP만 사용 (기본: true)

only_global_ips: false, // 테스트 환경에서는 false

};



autonat::Behaviour::new(local_public_key.to_peer_id(), config)

}
```

2. 다중 서버를 사용한 클라이언트

```
async fn setup_multi_server_client() -> Result<(), Box<dyn Error>> {

let mut swarm = create_swarm().await?;
```

```rust
// 여러 AutoNAT 서버 추가
let servers = vec![
("12D3KooWServer1...".parse()?, "/ip4/1.2.3.4/tcp/4001".parse()?),
("12D3KooWServer2...".parse()?, "/ip4/5.6.7.8/tcp/4001".parse()?),
("12D3KooWServer3...".parse()?, "/ip4/9.10.11.12/tcp/4001".parse()?),
];

for (peer_id, address) in servers {
swarm
.behaviour_mut()
.auto_nat
.add_server(peer_id, Some(address));
}

// 동적으로 서버 추가/제거
tokio::spawn(async move {
tokio::time::sleep(Duration::from_secs(60)).await;
// 새 서버 추가
let new_server: PeerId = "12D3KooWNewServer...".parse().unwrap();
let new_address: Multiaddr = "/ip4/13.14.15.16/tcp/4001".parse().unwrap();
// 주의: 실제로는 swarm에 대한 참조가 필요
// swarm.behaviour_mut().auto_nat.add_server(new_server, Some(new_address));
```

```
});


Ok(())

}
```

AutoNAT 서버 구현

1. 기본 서버

```
use libp2p::{autonat, identify, identity, noise, swarm::NetworkBehaviour, tcp, y


#[derive(NetworkBehaviour)]

struct ServerBehaviour {

identify: identify::Behaviour,

auto_nat: autonat::Behaviour,

}


impl ServerBehaviour {

fn new(local_public_key: identity::PublicKey) -> Self {

let config = autonat::Config {

// 서버 전용 설정

max_peer_addresses: 16,

throttle_clients_global_max: 50,
```

```
throttle_clients_peer_max: 5,

throttle_clients_period: Duration::from_secs(1),

only_global_ips: true, // 서버는 글로벌 IP만 허용

..Default::default()

};



Self {

identify: identify::Behaviour::new(identify::Config::new(

"/autonat-server/1.0.0".into(),

local_public_key.clone(),

)),

auto_nat: autonat::Behaviour::new(

local_public_key.to_peer_id(),

config,

),

}

}

}



#[tokio::main]

async fn main() -> Result<(), Box<dyn Error>> {

let mut swarm = libp2p::SwarmBuilder::with_new_identity()

.with_tokio()
```

```
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default,
)?
.with_behaviour(|key| ServerBehaviour::new(key.public()))?
.build();

// 공개 주소에서 리스닝
swarm.listen_on("/ip4/0.0.0.0/tcp/4001".parse()?)?;

println!("AutoNAT 서버 시작됨");
println!("피어 ID: {}", swarm.local_peer_id());

loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("서버 주소: {address}");
}
SwarmEvent::Behaviour(ServerBehaviourEvent::AutoNat(event)) => {
handle_server_autonat_event(event);
}
event => {
```

```
println!("서버 이벤트: {event:?}");
}
}
}
}

fn handle_server_autonat_event(event: autonat::Event) {
match event {
autonat::Event::InboundProbe(probe_event) => {
match probe_event {
autonat::InboundProbeEvent::Request { probe_id, peer, addresses } => {
println!("클라이언트 {peer}로부터 프로브 요청 받음 (ID: {probe_id:?})");
println!("테스트할 주소들: {addresses:?}");
}
autonat::InboundProbeEvent::Response { probe_id, peer, address } => {
println!("클라이언트 {peer}에게 성공 응답 전송 (ID: {probe_id:?}, 주소: {address})")
}
autonat::InboundProbeEvent::Error { probe_id, peer, error } => {
println!("클라이언트 {peer}에게 오류 응답 전송 (ID: {probe_id:?}, 오류: {error:?})")
}
}
}
_ => {} // 서버는 주로 인바운드 프로브만 처리
```

```
}

}
```

상태 모니터링 및 메트릭

1. NAT 상태 모니터링

```rust
use std::sync::{Arc, Mutex};

use std::collections::HashMap;


#[derive(Debug, Clone)]

struct NatStatusMonitor {

current_status: Arc<Mutex<autonat::NatStatus>>,

confidence: Arc<Mutex<usize>>,

status_history: Arc<Mutex<Vec<(std::time::Instant, autonat::NatStatus)>>>,

probe_stats: Arc<Mutex<ProbeStats>>,

}


#[derive(Debug, Default)]

struct ProbeStats {

total_probes: u64,

successful_probes: u64,

failed_probes: u64,
```

```
servers_used: HashMap<PeerId, u64>,
}

impl NatStatusMonitor {
fn new() -> Self {
Self {
current_status: Arc::new(Mutex::new(autonat::NatStatus::Unknown)),
confidence: Arc::new(Mutex::new(0)),
status_history: Arc::new(Mutex::new(Vec::new())),
probe_stats: Arc::new(Mutex::new(ProbeStats::default())),
}
}

fn update_status(&self, new_status: autonat::NatStatus, confidence: usize) {
let mut current = self.current_status.lock().unwrap();
let mut conf = self.confidence.lock().unwrap();
let mut history = self.status_history.lock().unwrap();

if *current != new_status {
history.push((std::time::Instant::now(), new_status.clone()));
// 최근 100개 기록만 유지
if history.len() > 100 {
history.remove(0);
```

```
}
}

*current = new_status;
*conf = confidence;
}

fn record_probe(&self, peer: PeerId, success: bool) {
let mut stats = self.probe_stats.lock().unwrap();
stats.total_probes += 1;
if success {
stats.successful_probes += 1;
} else {
stats.failed_probes += 1;
}

*stats.servers_used.entry(peer).or_insert(0) += 1;
}

fn get_status_summary(&self) -> String {
let status = self.current_status.lock().unwrap();
let confidence = self.confidence.lock().unwrap();
let stats = self.probe_stats.lock().unwrap();
```

```
format!(
"NAT 상태: {:?} (신뢰도: {})\n프로브 통계: 총 {}, 성공 {}, 실패 {}",
*status, *confidence, stats.total_probes, stats.successful_probes, stats.failed_
)
}
}

// 사용 예제
async fn run_monitored_client() -> Result<(), Box<dyn Error>> {
let monitor = NatStatusMonitor::new();
let mut swarm = create_swarm().await?;

// 주기적 상태 출력
let monitor_clone = monitor.clone();
tokio::spawn(async move {
let mut interval = tokio::time::interval(Duration::from_secs(30));
loop {
interval.tick().await;
println!("{}", monitor_clone.get_status_summary());
}
});
```

```
loop {
match swarm.select_next_some().await {
SwarmEvent::Behaviour(BehaviourEvent::AutoNat(event)) => {
match &event {
autonat::Event::StatusChanged { new, .. } => {
let confidence = swarm.behaviour().auto_nat.confidence();
monitor.update_status(new.clone(), confidence);
}
autonat::Event::OutboundProbe(probe_event) => {
match probe_event {
autonat::OutboundProbeEvent::Response { peer, .. } => {
monitor.record_probe(*peer, true);
}
autonat::OutboundProbeEvent::Error { peer: Some(peer), .. } => {
monitor.record_probe(*peer, false);
}
_ => {}
}
}
_ => {}
}
handle_autonat_event(event);
}
```

```
event => {
// 기타 이벤트 처리
}
}
}
}
```

2. 프로메테우스 메트릭 통합

```
use prometheus::{Counter, Gauge, Histogram, Registry};

struct AutoNatMetrics {
probe_requests_total: Counter,
probe_successes_total: Counter,
probe_failures_total: Counter,
current_confidence: Gauge,
probe_duration: Histogram,
nat_status: Gauge, // 0: Unknown, 1: Private, 2: Public
}

impl AutoNatMetrics {
fn new(registry: &Registry) -> Result<Self, prometheus::Error> {
let probe_requests_total = Counter::new(
```

```
"autonat_probe_requests_total",
"Total number of AutoNAT probe requests"
)?;
let probe_successes_total = Counter::new(
"autonat_probe_successes_total",
"Total number of successful AutoNAT probes"
)?;
let probe_failures_total = Counter::new(
"autonat_probe_failures_total",
"Total number of failed AutoNAT probes"
)?;
let current_confidence = Gauge::new(
"autonat_current_confidence",
"Current confidence level in NAT status"
)?;
let probe_duration = Histogram::with_opts(
prometheus::HistogramOpts::new(
"autonat_probe_duration_seconds",
"Duration of AutoNAT probes in seconds"
).buckets(vec![0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 30.0])
)?;
let nat_status = Gauge::new(
"autonat_nat_status",
```

```
"Current NAT status (0: Unknown, 1: Private, 2: Public)"
)?;

registry.register(Box::new(probe_requests_total.clone()))?;
registry.register(Box::new(probe_successes_total.clone()))?;
registry.register(Box::new(probe_failures_total.clone()))?;
registry.register(Box::new(current_confidence.clone()))?;
registry.register(Box::new(probe_duration.clone()))?;
registry.register(Box::new(nat_status.clone()))?;

Ok(Self {
probe_requests_total,
probe_successes_total,
probe_failures_total,
current_confidence,
probe_duration,
nat_status,
})
}

fn record_probe_request(&self) {
self.probe_requests_total.inc();
}
```

```
fn record_probe_success(&self, duration: f64) {
self.probe_successes_total.inc();
self.probe_duration.observe(duration);
}

fn record_probe_failure(&self, duration: f64) {
self.probe_failures_total.inc();
self.probe_duration.observe(duration);
}

fn update_confidence(&self, confidence: usize) {
self.current_confidence.set(confidence as f64);
}

fn update_nat_status(&self, status: &autonat::NatStatus) {
let status_value = match status {
autonat::NatStatus::Unknown => 0.0,
autonat::NatStatus::Private => 1.0,
autonat::NatStatus::Public(_) => 2.0,
};
self.nat_status.set(status_value);
}
```

```
}
```

테스트 및 디버깅

1. 로컬 테스트 환경

```
// 로컬 테스트를 위한 설정
fn create_test_config() -> autonat::Config {
autonat::Config {
timeout: Duration::from_secs(5),
boot_delay: Duration::from_secs(1),
retry_interval: Duration::from_secs(10),
refresh_interval: Duration::from_secs(30),
throttle_server_period: Duration::from_secs(5),
only_global_ips: false, // 로컬 테스트를 위해 false
confidence_max: 2, // 빠른 테스트를 위해 낮춤
..Default::default()
}
}

// 테스트용 서버와 클라이언트 동시 실행
#[tokio::test]
async fn test_autonat_local() {
```

```rust
// 서버 시작
let server_task = tokio::spawn(async {
let mut server_swarm = create_server_swarm().await.unwrap();
server_swarm.listen_on("/ip4/127.0.0.1/tcp/0".parse().unwrap()).unwrap();
// 서버 이벤트 루프
loop {
match server_swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("서버 리스닝: {address}");
}
_ => {}
}
}
});

// 잠시 대기 후 클라이언트 시작
tokio::time::sleep(Duration::from_secs(1)).await;

let client_task = tokio::spawn(async {
let mut client_swarm = create_client_swarm().await.unwrap();
client_swarm.listen_on("/ip4/127.0.0.1/tcp/0".parse().unwrap()).unwrap();
// 클라이언트 이벤트 루프
loop {
```

```
match client_swarm.select_next_some().await {
SwarmEvent::Behaviour(BehaviourEvent::AutoNat(event)) => {
println!("AutoNAT 이벤트: {event:?}");
}
_ => {}
}
}
});

// 테스트 실행
tokio::select! {
_ = server_task => {},
_ = client_task => {},
_ = tokio::time::sleep(Duration::from_secs(60)) => {
println!("테스트 완료");
}
}
}
```

2. 디버깅 도구

```
use tracing::{debug, info, warn, error};
```

```rust
fn setup_detailed_logging() {
tracing_subscriber::fmt()
.with_env_filter(
EnvFilter::from_default_env()
.add_directive("libp2p_autonat=debug".parse().unwrap())
.add_directive("libp2p_swarm=info".parse().unwrap())
)
.with_target(true)
.with_thread_ids(true)
.with_file(true)
.with_line_number(true)
.init();
}

// 상세한 이벤트 로깅
fn detailed_autonat_event_handler(event: autonat::Event) {
match event {
autonat::Event::OutboundProbe(probe_event) => {
match probe_event {
autonat::OutboundProbeEvent::Request { probe_id, peer } => {
info!("🔍 프로브 요청 전송: ID={:?}, 서버={}", probe_id, peer);
}
autonat::OutboundProbeEvent::Response { probe_id, peer, address } => {
```

```
info!("✅ 프로브 성공: ID={:?}, 서버={}, 공개주소={}", probe_id, peer, address);
}
autonat::OutboundProbeEvent::Error { probe_id, peer, error } => {
warn!("❌ 프로브 실패: ID={:?}, 서버={:?}, 오류={:?}", probe_id, peer, error);
}
}
}
autonat::Event::InboundProbe(probe_event) => {
match probe_event {
autonat::InboundProbeEvent::Request { probe_id, peer, addresses } => {
info!("📥 인바운드 요청: ID={:?}, 클라이언트={}, 주소수={}",
probe_id, peer, addresses.len());
debug!("주소 목록: {:?}", addresses);
}
autonat::InboundProbeEvent::Response { probe_id, peer, address } => {
info!("📤 인바운드 응답 성공: ID={:?}, 클라이언트={}, 주소={}",
probe_id, peer, address);
}
autonat::InboundProbeEvent::Error { probe_id, peer, error } => {
warn!("📤 인바운드 응답 실패: ID={:?}, 클라이언트={}, 오류={:?}",
probe_id, peer, error);
}
}
```

```
}
autonat::Event::StatusChanged { old, new } => {
info!("🔄 NAT 상태 변경: {:?} → {:?}", old, new);
}
}
}
```

실제 배포 고려사항

1. 프로덕션 설정

```
fn create_production_config() -> autonat::Config {
autonat::Config {
timeout: Duration::from_secs(30),
boot_delay: Duration::from_secs(15),
retry_interval: Duration::from_secs(90),
refresh_interval: Duration::from_secs(900), // 15분
throttle_server_period: Duration::from_secs(90),
use_connected: true,
confidence_max: 3,
max_peer_addresses: 16,
throttle_clients_global_max: 30,
throttle_clients_peer_max: 3,
```

```
throttle_clients_period: Duration::from_secs(1),
only_global_ips: true, // 프로덕션에서는 true
}
}
```

2. 오류 처리 및 복구

```
async fn robust_autonat_client() -> Result<(), Box<dyn Error>> {
let mut swarm = create_swarm().await?;
let mut retry_count = 0;
const MAX_RETRIES: u32 = 5;

loop {
match swarm.select_next_some().await {
SwarmEvent::Behaviour(BehaviourEvent::AutoNat(event)) => {
match event {
autonat::Event::OutboundProbe(autonat::OutboundProbeEvent::Error { error, .. })
match error {
autonat::OutboundProbeError::NoServer => {
warn!("사용 가능한 AutoNAT 서버가 없음");
// 새로운 서버 검색 로직
discover_new_servers(&mut swarm).await?;
}
```

```
autonat::OutboundProbeError::NoAddresses => {
warn!("다이얼백할 주소가 없음");
// 주소 발견 로직
trigger_address_discovery(&mut swarm).await?;
}
_ => {
retry_count += 1;
if retry_count >= MAX_RETRIES {
error!("최대 재시도 횟수 초과");
return Err("AutoNAT 프로브 실패".into());
}
}
}
}
_ => {
retry_count = 0; // 성공 시 재시도 카운터 리셋
handle_autonat_event(event);
}
}
}
event => {
// 기타 이벤트 처리
}
```

```
}

}

}


async fn discover_new_servers(swarm: &mut libp2p::Swarm<Behaviour>) -> Result<()

// DHT를 통한 AutoNAT 서버 검색 로직

// 또는 부트스트랩 노드에서 서버 목록 가져오기

Ok(())

}


async fn trigger_address_discovery(swarm: &mut libp2p::Swarm<Behaviour>) -> Resu

// Identify 프로토콜을 통한 외부 주소 발견

// 또는 수동으로 주소 추가

Ok(())

}
```

이러한 예제들을 통해 AutoNAT 프로토콜을 다양한 시나리오에서 효과적으로 활용할 수 있습니다. 각 예제는 실제 사용 사례에 맞게 수정하여 사용할 수 있습니다.